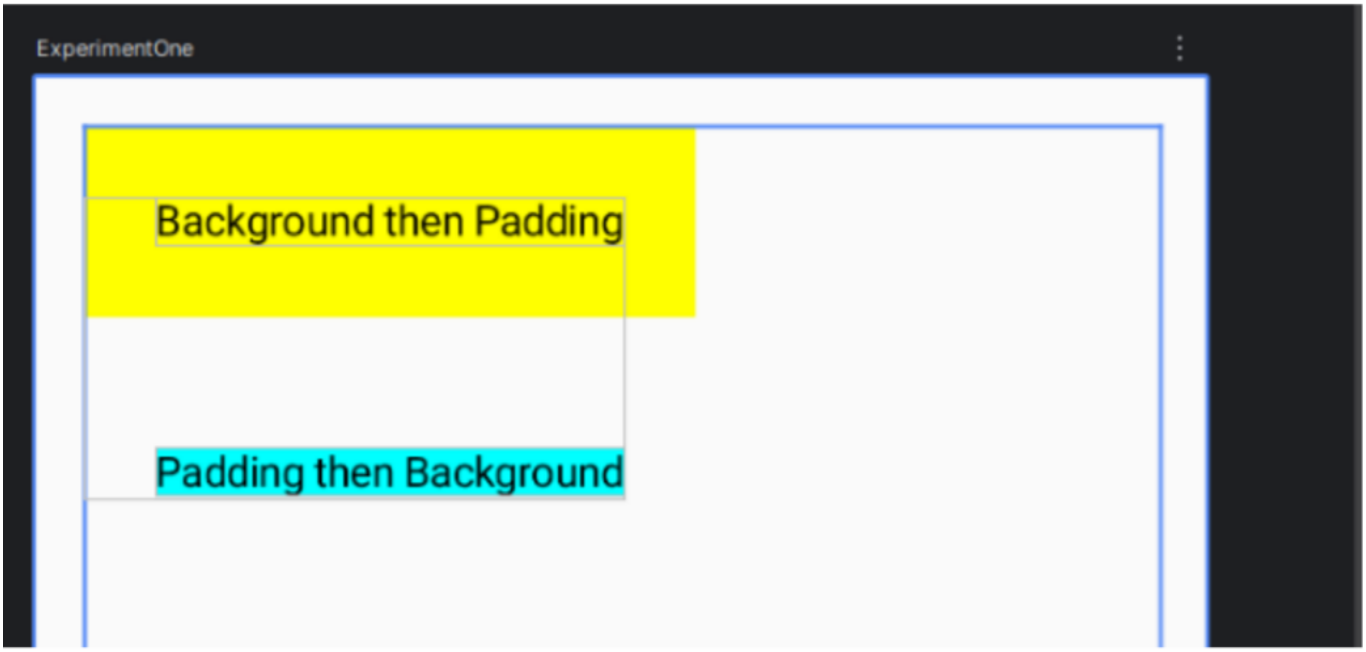
Task 3 — Modifiers, Spacing & Responsive Behavior Report

This report documents three experiments with Jetpack Compose Modifiers to analyze their impact on UI structure, responsiveness, and accessibility. Modifiers allow us to decorate or augment composables, and understanding their behavior—especially their order—is crucial for consistent Android development.

Perform 3 Experiments

Experiment 1: Modifier Order (Padding vs Background)



Mental Expectation

I expected that if the background is applied first, it will cover the entire area including the padding. If padding is applied first, the background will only appear behind the text.

Observations

Test A (Yellow) - `.background(Color.Yellow)` was called before `.padding(24.dp)` - The yellow color filled the text area first - Padding then extended the component boundaries

Test B (Cyan) - `.padding(24.dp)` was called before `.background(Color.Cyan)` - Padding created empty space first - Cyan color filled only the inner text area.

Conclusion

Modifier order matters significantly in Jetpack Compose because each modifier acts on the result of the previous one in the chain.

Experiment 2: fillMaxWidth() + Weight

Mental Expectation

I expected that by using `Modifier.weight()`, the available horizontal space would be divided proportionally. Specifically, with weights of **1f and 2f**, the blue box should be twice as wide as the red box.

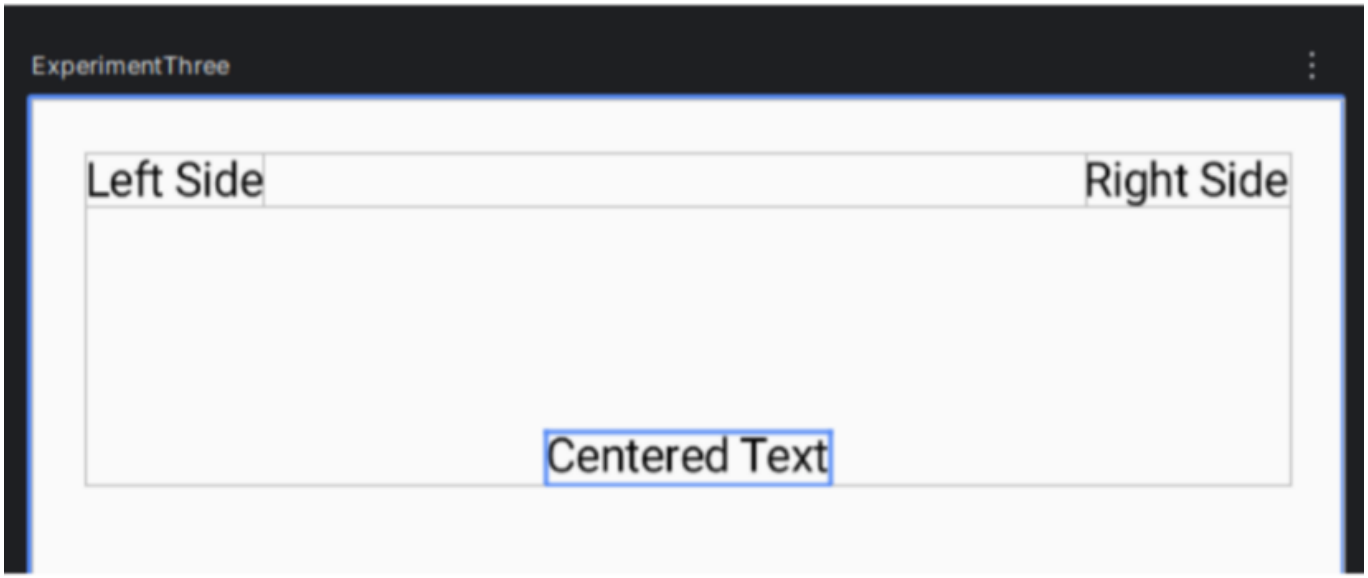
Observations

- The Row filled the maximum width of the screen
- The Red box took **one-third** of the width
- The Blue box took **two-thirds** of the width
- The ratio remains consistent across different screen sizes.

Conclusion

`Modifier.weight()` is a powerful tool for responsive layouts because it avoids hardcoded widths and allows elements to scale according to screen size.

Experiment 3: Alignment & Arrangement



Mental Expectation

I expected `Arrangement.SpaceBetween` to push the **Left Side** and **Right Side** texts to the edges while the centered text remains in the middle.

Observations

- Row distributed elements to left and right boundaries
- Centered text was aligned horizontally using `align` modifier

Conclusion

Combining Arrangement for group spacing and Alignment for individual positioning allows precise UI control without manual pixel offsets.

Documentation of Differences

Experiment 1: Background vs Padding (Order Matters)

Expectation - Background covers full area if applied first - Background covers only text area if applied after padding

Result - Yellow box: Background then Padding → color filled text boundaries first - Cyan box: Padding then Background → color filled only inner area

Modifier Order Explanation Modifiers are applied sequentially. Padding before background reduces the area available for the background.

Experiment 2: Proportional Scaling with Weight

Expectation - Blue box should be twice as wide as Red box based on 2:1 ratio **Result** - Blue box occupied two-thirds - Red box occupied one-third **Observation** The ratio remains consistent regardless of screen size.

Experiment 3: Alignment and Arrangement

Expectation - `Arrangement.SpaceBetween` pushes items to edges

Result - Left and Right labels pinned to edges - Centered text aligned in middle

Responsive Design Suggestions

To ensure layouts work across various Android devices:

1. Use Modifier.weight()

Avoid hardcoded widths like `200.dp`. Use weight so elements resize automatically.

2. Test on Multiple Devices

Use Compose Preview Device option to test on:

- Pixel 5
- Pixel Tablet

This helps detect layout problems early.

3. Prefer Arrangement over Hard Padding

Use:

- `Arrangement.spacedBy()`
- `Arrangement.SpaceBetween`

This ensures consistent spacing across screen sizes.

Accessibility Check (Mini)

Observed Issue

In Experiment 3:

- "Left Side" and "Right Side" texts are small
- Touch targets would be difficult if clickable.

Suggested Fix

- Use: Modifier.sizeIn(minWidth = 48.dp, minHeight = 48.dp)